College of Engineering and Technology

Web Engineering Department

Arab Academy for Science and Technology

PROJECT DOCUMENTATION

Web Engineering

Project Title:
Buzz Fly - Premium Flight & Hotel Booking Platform

Supervised By	Prof. Amr Fahmy
Course	Web Engineering
Semester	Semester 8
Academic Year	2025 - 2026

GitHub repo: https://github.com/Karimkimoz74/buzz-fly

Table of Contents

1. Project Team

1.1 Team Members

Full Name	College ID	Role / Responsibility	Email
Karim Mohamed Ismail	221004798	Front-end / Back-end	karimkimoz2003@gmail.com
Rawan Mohamed Ismail	221006089	Front-end / Back-end	rawan.m.ismail@gmail.com
Malak Wael Ghabn	221004011	Front-end / Back-end	malookaghabn@gmail.com
Omar Mohamed Elshafei	221005236	Front-end / Back-end	omar.mshafei@gmail.com
Ziad Amr Elmorshdy	221004883	Front-end / Back-end	ziadelmorshdy1@gmail.com
Mohamed Gasser Elsawah	221004214	Front-end / Back-end	mohamed.g.elsawah06@gmail.com

1.2 Sub-Group Distribution

Divide the team into sub-groups based on project layers or modules.

Sub-Group	Member(s)	Module / Layer	Key Deliverable
Frontend	All six members (Karim, Rawan, Malak, Omar, Ziad, Mohamed)	UI / UX, Bootstrap pages, custom SCSS utilities	Every page in the booking flow
Backend	All six members (Karim, Rawan, Malak, Omar, Ziad, Mohamed)	Firebase Auth + Firestore wiring, security rules	js/firebase/* data modules + bridges
Database	All six members (Karim, Rawan, Malak, Omar, Ziad, Mohamed)	Firestore collections + ownership rules	users / hotels / flights / bookings / notifications / support_tickets
DevOps / QA	All six members (Karim, Rawan, Malak, Omar, Ziad, Mohamed)	Git workflow, manual QA, Firebase Console deploys	Feature branches, PR reviews, manual click-through tests

2. Introduction & Problem Statement

2.1 Background

Provide context about the domain or field your web application addresses. Explain the current landscape, existing solutions, and why this project is needed.

Online travel booking is dominated by aggregator platforms that prioritize price search over curation. Premium travelers — the people booking longer stays, business cabins, and editorial destinations — are underserved by generic UIs that lump every tier into the same flow. The checkout experience on most platforms is also fragmented: travelers re-enter their identity and traveler details for every booking, with no in-app notification feed tying the journey together. Buzz Fly addresses these gaps with a curated, premium-feel web client backed by a single Firebase project that handles auth, data, and notifications behind one set of security rules.

2.2 Problem Statement

Describe the core problem your web application solves. Be specific and concise.

- No single curated platform for premium flights + hotels - most aggregators are price-first and miss the editorial layer (boutique stays, business-cabin upgrades).
- Existing checkout flows force the user to re-enter traveller details every booking; there is no profile-aware prefill across flights and hotels.
- Notification systems are usually limited to email; in-app feeds (booking confirmed, cancelled, 24-hour reminder) are rare on the web side.

2.3 Motivation

Explain why solving this problem matters and what impact the solution will have on users or the target audience.

Travel is one of the highest-trust categories in e-commerce. Building a premium, fast, and consistent experience across two devices - sharing the same booking data, the same notifications, the same traveller identity - is the kind of polish that turns a one-time visitor into a repeat customer.

3. Project Objectives

By the end of this project, the team will have achieved the following measurable goals:

- Ship a fully responsive multi-page web client (HTML + CSS + JS + Bootstrap 5.3) for premium flight and hotel bookings.
- Wire the entire app to a managed Firebase backend (Auth + Firestore) so the web client never needs a custom server while still enforcing per-user data ownership through security rules'.
- Implement the complete booking funnel for both flights and hotels: search -> results -> booking detail -> payment -> My Trips -> Trip Detail (with cancel).
- Build a per-user notification system with five triggers (welcome, welcome-back, booking confirmed, booking cancelled, 24-hour reminder) backed by an owner-only Firestore collection.
- Enforce data ownership with Firestore security rules so each user can only read and edit their own profile, bookings, and notifications.
- Optimise cold-cache first paint with preconnect + modulepreload hints on every Firebase page so the SDK downloads in parallel with HTML parsing.

4. Technology Stack

List every technology, library, and tool used in the project.

Layer	Technology / Tool	Purpose / Notes
Frontend Markup & Logic	Vanilla HTML, CSS, JavaScript (ES modules)	Page structure + page-level scripts (no framework)
Styling	Bootstrap 5.3 + custom SCSS (sass/main.scss)	Utility classes + branded .buzz-* tokens
Backend (BaaS)	Firebase Web SDK v10.14.1 - Auth + Firestore	Auth, data, security rules - no custom server
Database	Cloud Firestore (NoSQL document store)	6 collections: users / hotels / flights / bookings / notifications / support_tickets
ORM / ODM	None - Firebase SDK is the data layer	Modular reads/writes via getDoc / getDocs / addDoc / updateDoc
Authentication	Firebase Auth (Email/Password + Google OAuth)	Sign in, sign up, password reset, deferred-auth gate
Version Control	Git + GitHub (Karimkimoz74/buzz-fly)	Feature branches + PRs
Deployment	Firebase Hosting (planned for final demo)	Static-asset hosting from the same Firebase project
Testing	Manual browser click-through (Chrome DevTools)	Full booking flow + cancel + notifications verified per release
Additional Tooling	Bootstrap Icons 1.11 + Plus Jakarta Sans (Google Fonts)	Iconography + brand typography

5. System Architecture & Design

5.1 High-Level Architecture

Describe how the major components of your application interact (client, server, database, external APIs, etc.).

Two-tier architecture (no traditional middle tier). The web client (static HTML + JS) speaks directly to Firebase, which provides Auth + Firestore + (planned) Storage. There is no custom server. Security and ownership are enforced by Firestore rules — every read and write is gated by request.auth.uid against the doc's userId field.

5.2 Frontend Architecture

Describe the component structure, routing strategy, and state management approach.

- Pages: home (index.html) + auth (signin/signup/forgot/logout) + hotels (results, details) + flights (search-results, details) + payment + profile (profile, settings, my-trips, trip-detail) + support (contact, help, notifications) + legal pages.
- State management: vanilla page-local state plus cross-page sessionStorage bridges (payment-bridge, flight-bridge, trip-detail-bridge) for handing off complex objects.
- Routing: classic multi-page navigation. Shared navbar + sidebar + footer injected via fetch in js/main.js. Each page loads its own module via script type=module.

5.3 Backend Architecture

Describe the API design, middleware, and server structure.

- Firebase serves as the backend (BaaS) - no Express, no custom routes. Data access is wrapped in 13 modules under js/firebase/ (auth, bookings, flights, hotels, notifications, support, constants, init, plus bridges for cross-page state).
- Security model: Firestore rules enforce ownership (request.auth.uid == resource.data.userId) for bookings, notifications, and the user's own profile.
- Cross-cutting helpers: avatar-cache (pub/sub + localStorage), navbar-auth (visibility toggles + sign-out delegation), payment-bridge / flight-bridge / trip-detail-bridge (sessionStorage handoffs between pages).

5.4 Database Schema

Describe the main entities, relationships, and key fields.

Six top-level collections. users/{uid} stores the profile (fullName, email, phoneNumber, passportNumber, dateOfBirth, gender, country, avatar). hotels/{id} and flights/{id} are public read-only catalogues seeded by an admin script. bookings/{autoId} is owner-only with travelDate, optional returnDate (omitted for one-way flights - its presence is the round-trip flag), segments[], traveller{}. notifications/{autoId} is per-user with type (booking|system) + isRead + relatedBookingId?. support_tickets/{autoId} is create-only - admins read via the Firebase Console.

6. Project Steps & Phases

Outline each development phase with its tasks and expected outputs.

Phase	Step / Task	Expected Output / Deliverable
Phase 1	Requirements gathering, Figma mockups, page inventory	Figma designs + page-by-page assignment matrix
Phase 2	Firestore schema design + hotel / flight catalogue seed via admin script	Documented collection contracts, seeded hotels + flights catalogues
Phase 3	Project scaffolding, Bootstrap 5.3 migration, Git feature-branch workflow	Initialised repo, SCSS pipeline, branch strategy, partial-injection in js/main.js
Phase 4	Firebase data layer - auth.js, hotels.js, flights.js, bookings.js, notifications.js	13 modules under js/firebase/ + payment + flight + trip-detail bridges
Phase 5	Page-by-page UI implementation - each team member owns specific pages	Hotel + flight + auth + profile + my-trips + contact + notifications pages live
Phase 6	Authentication, profile, deferred-auth gate, Firestore security rules	Sign-in flow, owner-only rules at docs/firestore.rules, avatar instant-paint
Phase 7	Manual testing - booking flow, round-trip leg splitting, cancel, notifications	Bug fixes shipped per session; mojibake, dropdown overflow, etc.
Phase 8	Final report, deployment to Firebase Hosting, in-class demo	This document + a live URL + presentation slides

7. Group Task Distribution

Assign each feature or module to a team member. Update the Status column as the project progresses.

Module / Feature	Assigned Member(s)	Sub-Group	Status
Auth pages (signin / signup / forgot / logout)	Karim	Full-Stack	Done
Home page + Navbar + Footer partials	Karim	Full-Stack	Done
Sidebar + Profile + Edit Profile + Settings + My Trips+ Log Out	Rawan	Full-Stack	Done
Hotel Results + Hotel Details	Malak	Full-Stack	Done
Flight Search Results + Flight Details + Confirm Payment	Mohamed	Full-Stack	Done
Help Center + Notifications page+ Privacy Policy + Terms of Services	Ziad	Full-Stack	Done
Contact Us + About Us + Trip Detail	Omar	Full-Stack	Done
Firebase wiring + security rules (cross-cutting)	Team-wide	Full-Stack	Done

8. Development Timeline

Adjust weeks to match your actual course schedule.

Week	Phase / Task	Responsible Sub-Group	Deliverable
9	Kickoff - Figma mockups, page inventory, role split	All	Designs + assignment matrix
14	Firestore schema agreement with mobile team + catalogue seed	Backend + Database	Collection contracts + seeded hotels/flights
9	Scaffolding, Bootstrap migration, Git workflow	All	Working dev environment + first PRs
14	Firebase data layer + Auth + sign-in/up wiring	Backend	js/firebase/* modules + auth pages live
10	Page-by-page UI implementation - Hotels, Flights, Profile, etc.	Frontend	Core pages live against Firestore
11	Booking flow integration - payment, My Trips, Trip Detail	All	End-to-end booking working
12	Notifications system + 24h reminder + cancel flow + bug fixes	All + QA	Polished build + test pass
15	Deployment to Firebase Hosting, final report, in-class demo	DevOps + All	Live app + this report + slides

9. Full Project Documentation

9.1 Features & Functional Requirements

List every feature the application must support.

- Email + Google authentication with deferred-auth gating on booking pages; profile completion onboarding flow for new accounts.
- Hotel booking funnel - destination search with autocomplete, results pipeline (destination filter -> roomType scope -> chip -> paginate), detail page with 5-image carousel, room-tier picker with reactive totals, Book-for-another-traveller toggle.
- Flight booking funnel - round-trip / one-way modes, results page with Best Value / Fastest / Cheapest chips, plane-on-line UI, per-leg independent cabin pickers with price deltas relative to the selected class, baggage allowance derived from the outbound cabin.
- Shared payment view - formatters for cardholder / card number / expiry / CVV, live validation, addDoc to bookings/{autoId} with Timestamps, confirmation modal, history-replace navigation to My Trips.
- My Trips with leg-splitting (round-trip flights expand to two date-keyed cards), Trip Detail with modal-confirmed cancellation, in-app notifications feed with five triggers (welcome / welcome-back / booking confirmed / cancelled / 24h reminder), and a Contact Us form that writes owner-tagged support tickets.

9.2 Non-Functional Requirements

- Performance - preconnect + modulepreload hints on every Firebase page so the SDK downloads in parallel with HTML parsing (~300-500 ms saved on cold cache). Avatar instant-paint via localStorage prime - zero flash.
- Security - Firestore rules enforce ownership (auth.uid == resource.data.userId) on users / bookings / notifications. Support tickets are create-only. Card details are never persisted. Hotels / flights are read-only from the client (admin-seeded).
- Scalability - BaaS architecture means Firebase handles auto-scaling. Client-side queries are kept to single-equality filters where possible to avoid composite indexes.
- Accessibility - semantic HTML, ARIA labels on icon-only buttons (notifications, account dropdown, swap, etc.), keyboard-navigable Bootstrap form controls.
- Responsiveness - Bootstrap 5.3 mobile-first grid, tested 320 px - 1440 px. Sidebar collapses to a hamburger menu on < 992 px.

9.3 API Documentation

Buzz Fly is BaaS-backed - instead of REST endpoints, the client speaks to Firestore via the Firebase Web SDK. The table below lists the main collection operations.

Collection Path	Operation	Description	Auth Required	Owner-Only
users/{uid}	getDoc / setDoc(merge)	Read + write the user's own profile	Yes	Yes (uid == auth.uid)
hotels/{id}	getDocs	Read the public hotel catalogue	No	No (public)
flights/{id}	getDocs (where origin / dest)	Read flights by route (date-less)	No	No (public)
bookings/{autoId}	addDoc / getDocs / updateDoc	Create + list user's bookings + cancel	Yes	Yes (userId match)
notifications/{autoId}	addDoc / getDocs / updateDoc	Per-user feed - welcome / booking / reminder	Yes	Yes (userId match)
support_tickets/{autoId}	addDoc	Contact Us submissions - create-only from the client	Yes	Yes on write; reads blocked

9.4 UI/UX Design Notes

Describe the design language, color palette, typography, and key user flows.

- Colour palette - Primary green gradient #006D33 -> #1FAF5A (brand), accent purple #4F00D0 (system / cancel actions), neutral background #F8F9FA, body text #191C1D.
- Typography - Plus Jakarta Sans (Google Fonts), 16 px base, 1.6 line height; weights 400 / 500 / 600 / 700 / 800.
- Key user flows - Home -> Hotels/Flights tab -> Search -> Results -> Detail -> Confirm -> Payment -> My Trips -> Trip Detail (cancel). Notifications surface in the navbar bell after every booking event.

Figma file - internal team link; design tokens replicated in sass/main.scss as $dark-green / $gradient-green / $dark-purple etc.

9.5 Testing Plan

Test Type	Tool / Method	Scope	Pass Criteria
Manual functional testing	Chrome DevTools - click-through	Full booking flow (search -> detail -> payment -> My Trips -> cancel)	Every flow lands on the right screen with the right data
Integration testing	Firebase Console - Firestore data viewer	Verify addDoc writes the expected schema	Every doc carries userId == auth.uid and the required fields
Cross-device responsiveness	Chrome DevTools device toolbar	Mobile 320 px <-> desktop 1440 px	No layout breaks, navbar collapses correctly under 992 px
Auth + rule testing	Firebase Rules simulator + manual	Owner-only reads on bookings + notifications	Unauthorised read attempts return permission-denied

9.6 Deployment & DevOps

- Hosting platform - Firebase Hosting (planned for final demo). Project ID buzz-fly-mobile-2a08c, region me-central1.
- CI/CD - manual deploy via firebase deploy at milestone branches; future work includes GitHub Actions running lint on PRs.
- Environment configuration - Firebase web config is hardcoded in js/firebase/firebase-init.js (public per Firebase guidance); security is enforced by rules, not by hiding the apiKey.
- Domain - final URL set via Firebase Hosting custom domain once deployed.

10. Submission Checklist

- ☐ System architecture diagram completed
- ☐ Database ERD finalized and approved
- ☐ All API endpoints documented (Swagger / Postman collection)
- ☐ Frontend pages implemented and responsive
- ☐ Backend API tested and returning correct responses
- ☐ Authentication and authorization working correctly
- ☐ Frontend and backend fully integrated
- ☐ Database seeded with sample data
- ☐ Unit tests written and passing
- ☐ End-to-end tests covering critical flows
- ☐ Application deployed to production URL
- ☐ Environment variables secured (no secrets in repo)
- ☐ Code reviewed and merged to main branch
- ☐ Final project report completed and formatted
- ☐ Presentation slides prepared
- ☐ Live demo rehearsed by all team members